Part 1

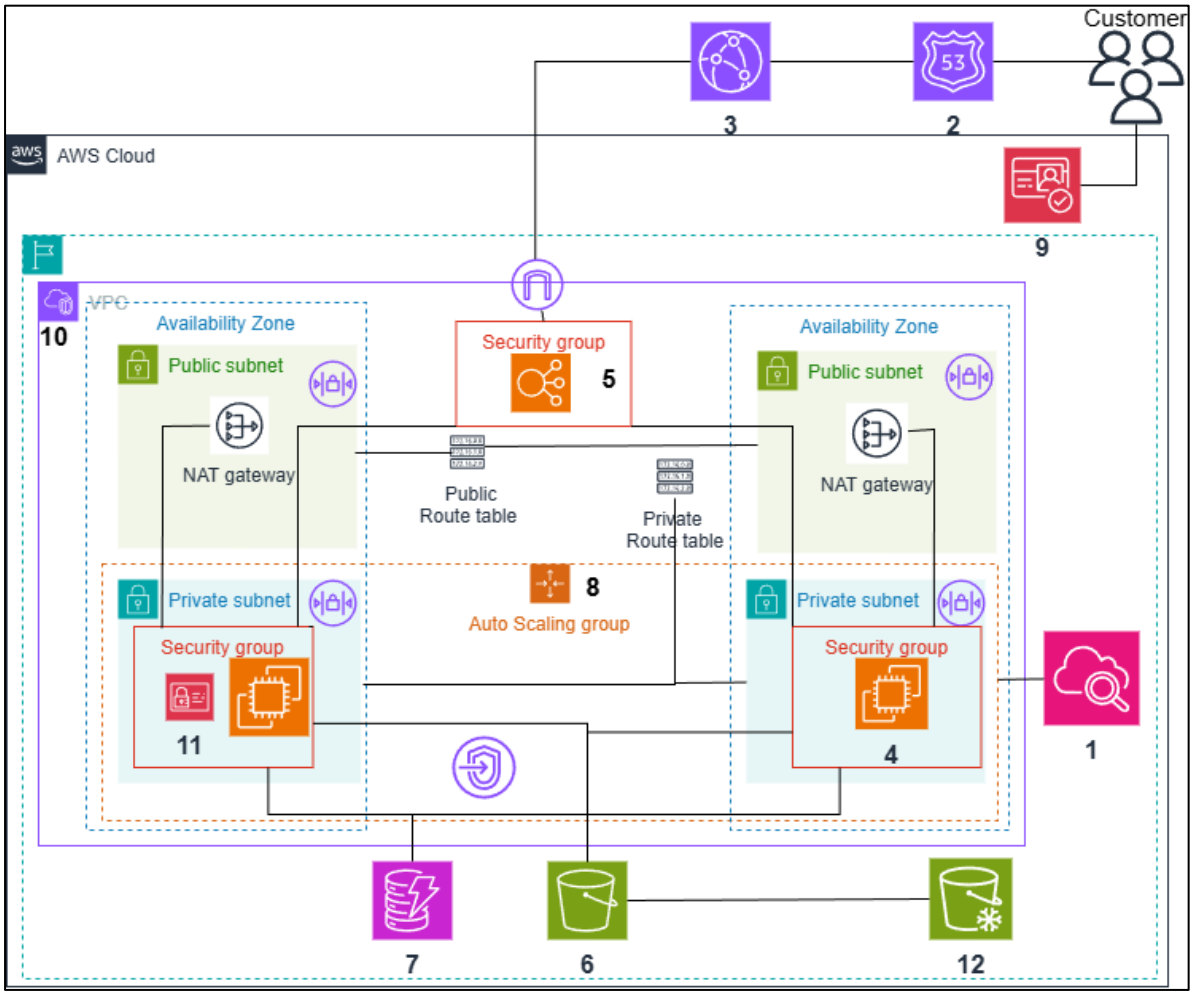

Figure 1: AWS Architecture for a public webservice called “ArtAI”, where users can upload images and receive an AI-generated variation. **Legend for Figure 1:** 1. AWS Cloud Watch, 2. AWS Route 53, 3. AWS CloudFront, 4. AWS EC2, 5. AWS Elastic Load Balancing, 6. AWS S3, 7. AWS Dynamo DB, 8. AWS Auto Scaling, 9. AWS Cognito, 10. AWS VPC, 11. AWS Identity Access Management, 12. AWS S3 Glacier

Description:

1. **AWS Cloud Watch:** Fixing a CloudWatch Alarm on Request Count greater than 10,000 in a single day and attaching a Simple Notification Service topic would send an email notification once the threshold crosses.
2. **AWS Route 53:** It manages domain names, helps in domain to IP translation, manages traffic and routes traffic to healthy servers which lowers latency and increase performance.
3. **AWS CloudFront:** It accepts HTTPS requests and do encryption, delivers content from the nearest edge location, routes traffic to the nearest edge location which helps in load balancing, lowers latency and improves performance.

4. **AWS EC2:** It creates a virtual server where I deploy the model and provides compute(CPU, RAM, Memory) for running my model.

Why EC2? It provides persistent workload for long running webservice and gives full freedom of choice and management of Operating System, instance types, Networking and security, and database. It also saves cost because of self-scaling.

5. **AWS Elastic Load Balancing**: Spreads requests evenly to all healthy EC2 instances. It lowers the latency and enhances the performance during high traffic.
6. **AWS S3 (Simple Storage Service)**: It is an object storage which is good for static content for example images uploaded by the users, gives highly durability (11 nines) and availability, provides virtually unlimited storage for scaling. It's cost effective, provide backups (transferring files to S3 Glacier). It also assists versioning and cross region replication.
7. **AWS Dynamo DB**: It stores metadata in database, for example: User data, upload date and timestamps, description of image, S3 file path. It provides low latency, handles millions of requests, automatically scales and gives on-demand backup.
8. **AWS Auto Scaling**: It helps in adding and removing the EC2 instances on the basis of traffic load so that there is always enough compute capacity. It also saves cost.
9. **AWS Cognito:** It helps in user authentication and authorization by enabling user to sign up and login. It uses email, password to access the site.
10. **AWS VPC (Virtual Private Cloud):** It provides private and an isolated network inside AWS where I can manage IP, subnets, route tables, firewalls and ensure connectivity for my resources.
    To increase the availability of the webservice I have deployed the model in two availability zones and placed the model in private subnets to prevent public access. I have attached security groups and Network Access Control List to enhance the instance and subnet level security. VPC Endpoints have been used to securely connect EC2 in private subnet with AWS S3 and Dynamo DB.
11. **AWS IAM (Identity Access Management):** I have attached IAM role to EC2 for performing operations like read and write on S3 and Dynamo DB. I have also used IAM users to control who can reach out to my AWS resources and their actions.
12. **AWS S3 Glacier:** Files from AWS S3 are moved to AWS S3 Glacier using S3 Lifecycle rules and thus it protects the data and helps in backing up. It also provides 11 nines durability.

How this application works to back up the data automatically when it's uploaded?

The images uploaded by the users are stored in AWS S3 and these images are moved from AWS S3 to AWS S3 Glacier in batches via lifecycle rules. The files stored in the glacier work as a backup which can be retrieved in the future. I have also used AWS Dynamo DB which contains image's metadata. When a user wants the image later, the combination of AWS Dynamo DB and S3 would help to retrieve the exact image the user had uploaded.

Part-2

Task A

Objective: The goal of Task A is to set up the WordFreq application in the AWS Academy Learner Lab. This includes deploying the necessary AWS resources along with ensuring that the application can read and process text files and stores the results.

WordFreq Application: The functionality of this application it to count words in a text file. It generates the top ten most frequent words found in the file and can also process multiple such files sequentially.

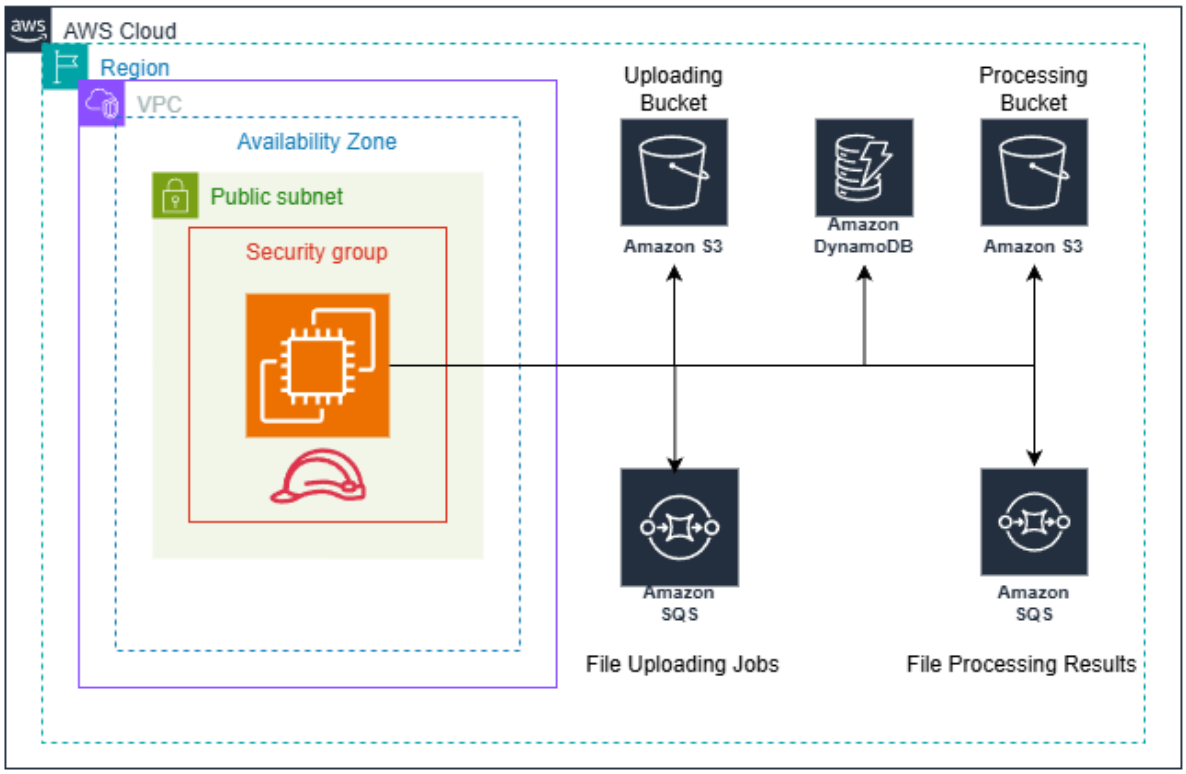

Figure 2: AWS Architecture for WordFreq Application

AWS Components Used in WordFreq

1. **AWS EC2:** It provides the virtual server which runs the WordFreq application. The application code is written in Go language and is deployed on an Ubuntu EC2 instance.
2. **Amazon S3:** There are two S3 buckets used by this application:
   - **Uploading bucket**: In this bucket, the original text files are uploaded and stored.
   - **Processing bucket**: In this bucket, the files are copied from the uploading bucket which are ready to be used by the worker application.

3. **Amazon Dynamo DB:** It stores the result given by the EC2 instance, which includes the top 10 most frequent words taken out from each processed text files.
4. **Amazon SQS:** There are two SQS queues used in the application:
    - **File Uploading Jobs**: This queue holds messages whenever new text files are uploaded in the processing bucket.
    - **File Processing Results**: This queue collects the result after each text file is processed by the app.

How the WordFreq application works?

1. **Uploading of text files to S3 Processing Bucket**

A preconfigured event notification gets triggered whenever a text file is uploaded to the bucket. This event notification is connected to the AWS SQS **File Uploading Jobs** Queue and S3 sends a message to the SQS queue whenever a text file is uploaded. The message contains **Bucket Name**, **File Path and Size, Event name** and **time** etc.

2. **Work of SQS File Processing Jobs Queue**

In **File Uploading Jobs** Queue, messages sent by the **Processing bucket** are queued in a reliable "First in First Out " like order. It ensures files are processed one by one and without loss.

3. **Work of EC2 in file processing**

The EC2 instance regularly checks the **File Uploading Jobs** Queue for the message and when a message is found, EC2 learns the location of the text file from the message, **the IAM Role** gives bucket access and download the text file from the **Processing Bucket** and processes the text file.

Once the processing is done, the message is deleted from the **File Uploading Jobs** queue. This ensures that each file is processed only one time which also avoids duplicate processing during Auto Scaling. Over all this prevents other EC2 instances from picking the same file.

4. **SQS File Processing Results Queue**

Once the analysis gets finished, EC2 send the output to the **File Processing Results** Queue. The output stores information like file name, Word Frequency results, timestamps etc.

5. **Role of Dynamo DB**

It is a database which reads messages from the **File Processing Results** queue and write the output in DynamoDB. It is the final storage location of the results.

6. Complete Workflow Summary

S3 Upload -------> SQS File Processing Jobs Queue --------> EC2 Processing-------> SQS File Processing Results Queue ---------> DynamoDB Storage

Task B

Objective: Design and Implement Auto-scaling

CloudWatch Configuration :

1. **Metric Selection :**

   Metric name – **ApproximateNumberofMessagesVisible**
   Queue Name - **wordfreq-jobs** (File Processing Jobs Queue)

Why this metric:

1. It tells us about how many files are standing in the File Processing Jobs queue to be processed and in this way, it directly reflects the workload of the instances.
2. It helps in auto scaling because when the queue is long , the system can add instances and can remove instances when the queue is short, a cost-effective solution.

2. **CloudWatch Alarms :**

Statistics: Average
Why average? The average statistic gives the mean of the metric count over the selected period of time and it provides a stable and a balanced view of the queue load. It also prevents unnecessary adding instances due to short term spikes.

Alarms:
I have configured the following alarms.

Name	State	Conditions	Actions
removeinstance	In alarm	ApproximateNumberOfMessagesVisible < 5 for 1 datapoints within 1 minute	Actions enabled
addinstance	OK	ApproximateNumberOfMessagesVisible > 15 for 1 datapoints within 1 minute	Actions enabled

Fig 3: CloudWatch Alarm Dashboard

1. **removeinstance:** This alarm triggers when the metric count is below 5 messages in the File Processing Jobs Queue. The metric is evaluated in every 1 minute. The purpose of this alarm is to trigger scaling in operation in the auto scaling group to prevent unnecessary EC2 instances from running.
2. **addinstance:** This alarm triggers when the metric count is above 15 messages in the File Processing Jobs Queue. The metric is evaluated in every 1 minute. The purpose of this alarm is to trigger scaling out operation in the auto scaling group in response of many pending messages in the job queue.

Auto Scaling Group (ASG) Configuration:

1. **ASG Name:** workfreqdev

2. **Launch Template:** The launch template used here is **workfreqdev** which was created using the original EC2 instance **wordfreq-dev.**

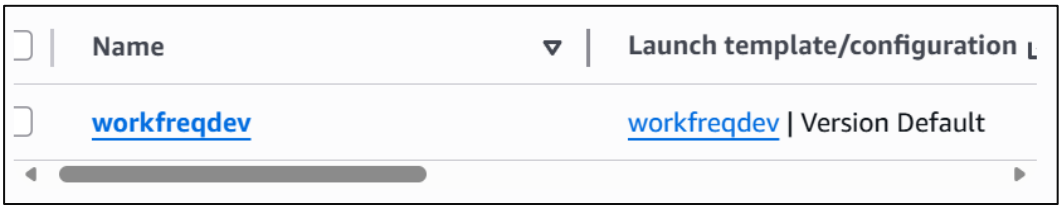

Fig 4: Auto Scaling Group Dashboard

3. **Minimum Instances:** 1
   It ensures that there is always at least one EC2 instance running to process the text file whenever the system is live.

4. **Maximum Instances:** 9
   It ensures that excessive instance creation doesn't happen which saves costs.

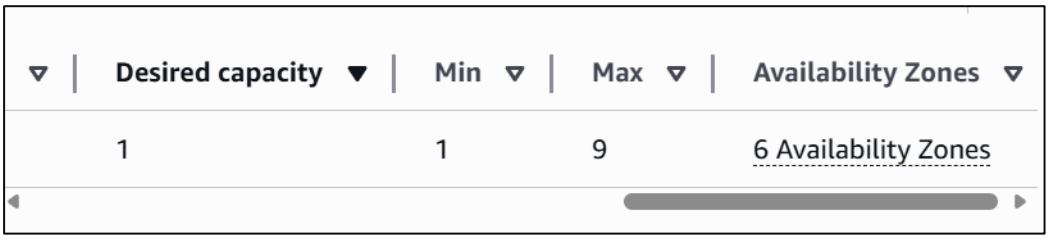

Fig 5: Scaling Configuration

5. **Scaling Policies:**
   1. I have used **Dynamic Scaling** because it automatically adds or removes instances based on the number of the messages in the SQS queue. It manages the spikes efficiently and at the same time it is cost effective because it runs only the necessary instances.

2. I have implemented the policy using **Simple Scaling**. Here, each CloudWatch alarm directly triggers a scaling action which can add or remove instances. It decreases the possibility of launching too many instances too quickly.

3. There is a **cooldown period of 2 min** between scaling operations.

Specific policies which govern scaling in and scaling out operations:

- **workfreqaddcapacity:** This policy is attached to **addinstance** cloud watch alarm. It adds one EC2 instance when alarm is triggered and then wait for 2 minutes before any next scaling action.

- **wordFreqRemoveCapacity:** This policy is attached to **removeinstance** cloud watch alarm. It removes one EC2 instance when alarm is triggered and then wait for 2 minutes before any next scaling action.

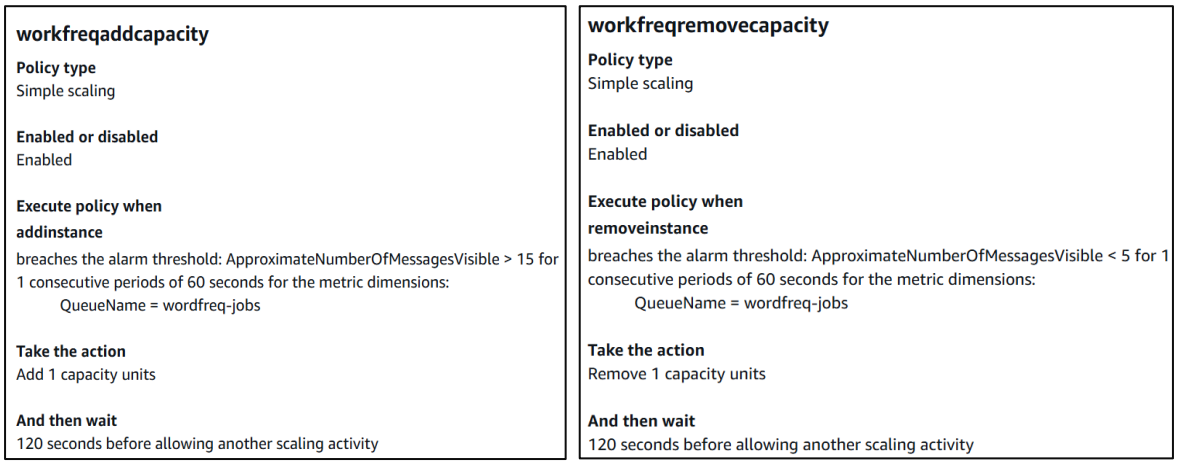

Fig 6 – 7: Scaling Policies

6. **Termination Policies:**
   The policy used is **NewestInstance** which ensures that during scaling in, the most recently launched EC2 instance is terminated first.

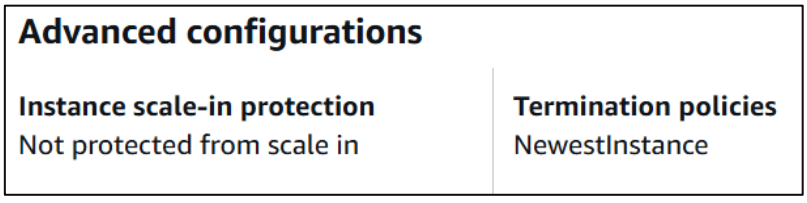

Fig 8: Termination Policy

Auto scaling Workflow with CloudWatch

1. When S3 processing bucket receives a text file, a SQS message is added to the **File Processing Jobs Queue**.
2. Greater the number of the files in the queue, greater is the workload.
3. CloudWatch constantly tracks the SQS metric count and when thresholds are crossed, CloudWatch sends a signal to the ASG.
4. ASG adds an instance when the metric count exceeds the upper threshold, and ASG removes an instance when the metric count dip below the lower threshold.
5. EC2 processes a file and deletes the SQS **File Uploading Jobs queue** message.
6. The above cycle repeats.

Task-C

Objective: Perform Load Testing

Test Preparation:

- **Creating the Test Files**

  -Procured 120 text files from the Blackboard.

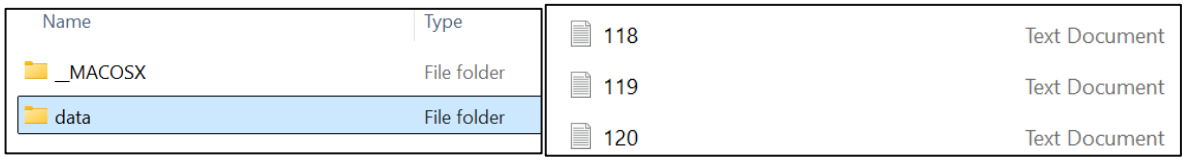

Fig 9-10: Picture from file explorer

- **Preparing S3 Buckets**

  -Uploading bucket (**as-wordfreq-nov25-uploading**) contains 120 files.

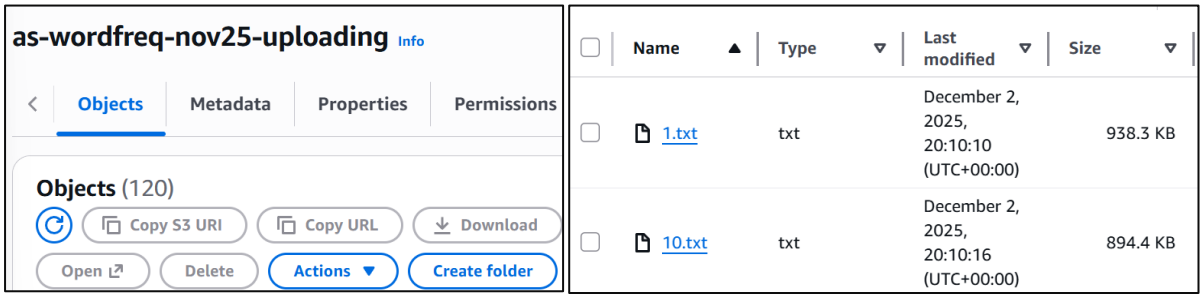

Fig 11-12: Screenshots from S3 uploading bucket showing 120 files

- **Stopping the Original Worker Instance (wordfreq-dev)**

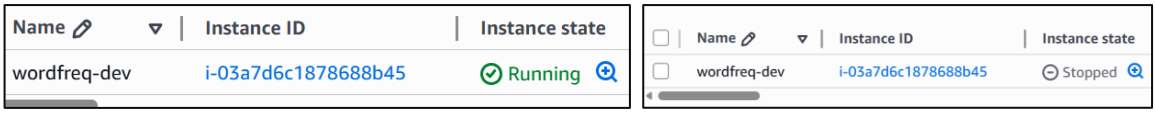

Fig 13: EC2 dashboard showing different Instance State

Test Execution Steps:

- **SSH connection to an Auto Scaling Group Instance**

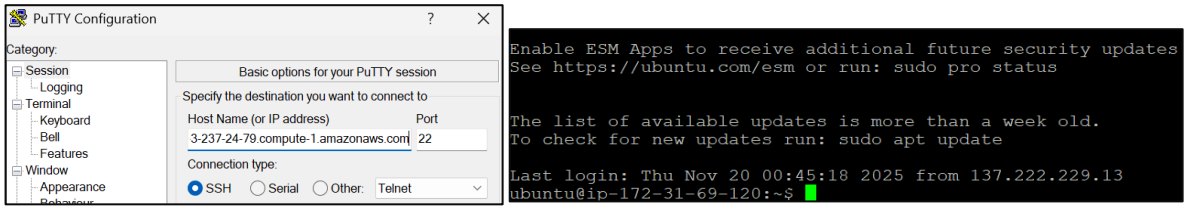

Fig 14-15: Putty Interface and Command Line showing SSH connection

- **Copying files from uploading bucket (as-wordfreq-nov25-uploading) to processing bucket (as-wordfreq-nov25-data-processing).**

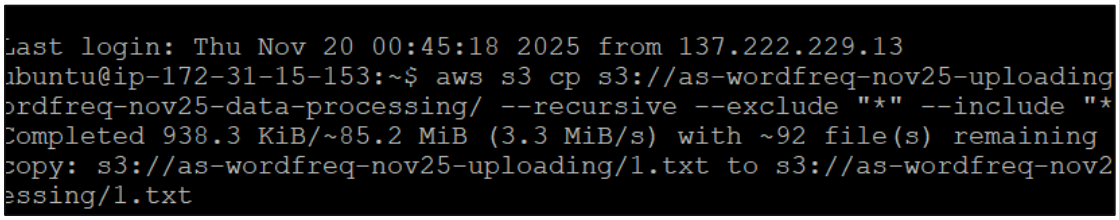

Fig 16: Command Line showing copying between buckets

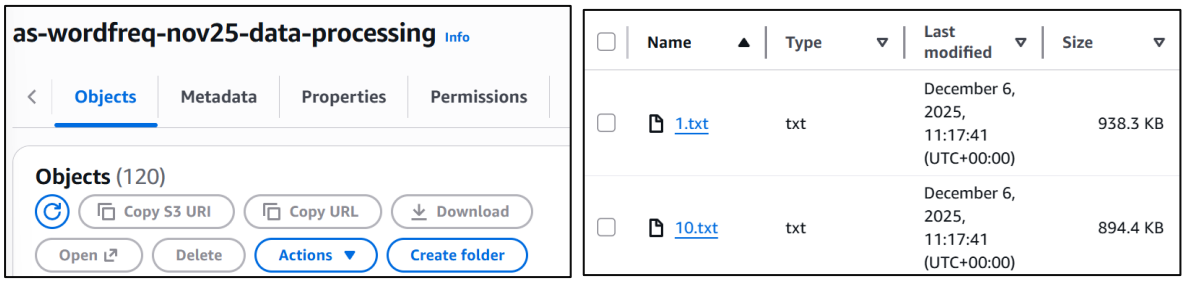

Fig 17-18: Screenshots from processing bucket showing 120 objects.

Monitoring Auto Scaling Behavior:

- **Scale Out Behavior (Add instance):**
    1. After copying 120 text files into the processing bucket (**as-wordfreq-nov25-data-processing)**, the number of messages in the **File Uploading Jobs queue** spiked instantly.

2. When the number of messages in the **File Uploading Jobs queue** crossed the upper limit threshold, the **addinstance** alarm went into ALARM state.
3. The cloud watch alarm triggered the ASG scaling out policy which led to the launch of EC2 instances one by one in the interval of 2 minutes.

- **Scale In Behavior (Remove Instances):**
    1. When the number of messages in the **File Uploading Jobs** Queue crossed the lower limit threshold , then the **removeinstance** alarm went into ALARM state.
    2. The cloud watch alarm triggered the ASG scaling in policy which led to the termination of EC2 instances one by one in the interval of 2 minutes.

Screenshots:

- **Copied files in the S3 bucket**

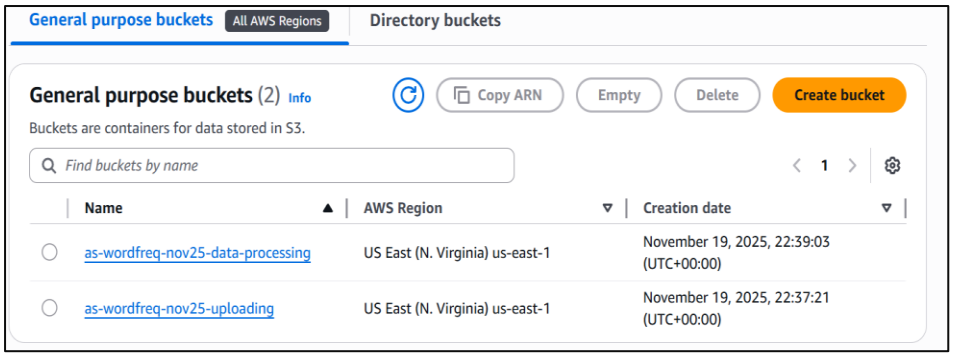

Fig 19: S3 Bucket Dashboard

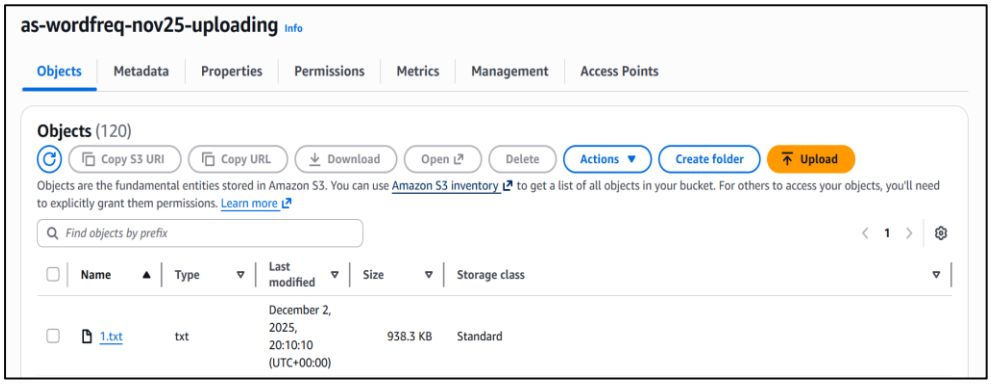

Fig 20: S3 Uploading bucket showing 120 text files

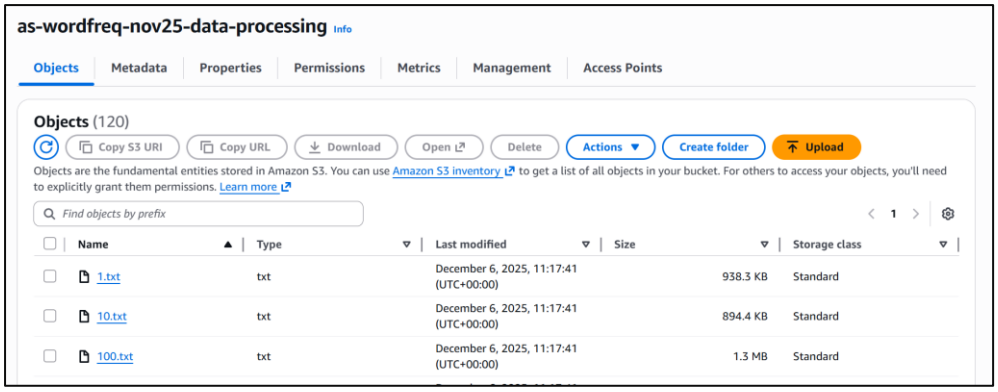

Fig 21: S3 processing bucket showing 120 text files

- **SQS Queue Status**

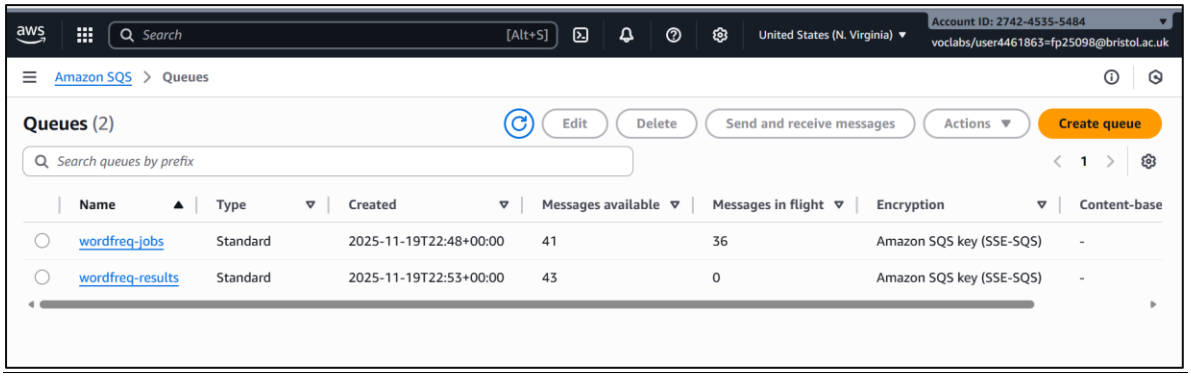

Fig 22: SQS showing message status

- **Auto Scaling Group**

Successful	Terminating EC2 instance: i-0c22cc120487a2681	At 2025-12-06T11:33:54Z a monitor alarm removeinstance in state ALARM triggered policy workfreqremovecapacity changing the desired capacity from 3 to 2. At 2025-12-06T11:33:56Z an instance was taken out of service in response to a difference between desired and actual capacity, shrinking the capacity from 3 to 2. At 2025-12-06T11:33:56Z instance i-0c22cc120487a2681 was selected for termination.	2025 December 06, 11:33:56 AM +00:00
Successful	Terminating EC2 instance: i-054b6e9f1d0ad6a15	At 2025-12-06T11:30:54Z a monitor alarm removeinstance in state ALARM triggered policy workfreqremovecapacity changing the desired capacity from 4 to 3. At 2025-12-06T11:31:03Z an instance was taken out of service in response to a difference between desired and actual capacity, shrinking the capacity from 4 to 3. At 2025-12-06T11:31:04Z instance i-054b6e9f1d0ad6a15 was selected for termination.	2025 December 06, 11:31:04 AM +00:00
Successful	Launching a new EC2 instance: i-054b6e9f1d0ad6a15	At 2025-12-06T11:26:17Z a monitor alarm addinstance in state ALARM triggered policy workfreqaddcapacity changing the desired capacity from 3 to 4. At 2025-12-06T11:26:26Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 3 to 4.	2025 December 06, 11:26:28 AM +00:00
Successful	Launching a new EC2 instance: i-0c22cc120487a2681	At 2025-12-06T11:23:17Z a monitor alarm addinstance in state ALARM triggered policy workfreqaddcapacity changing the desired capacity from 2 to 3. At 2025-12-06T11:23:23Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 2 to 3.	2025 December 06, 11:23:25 AM +00:00

Fig 23: Activity in Auto Scaling Group

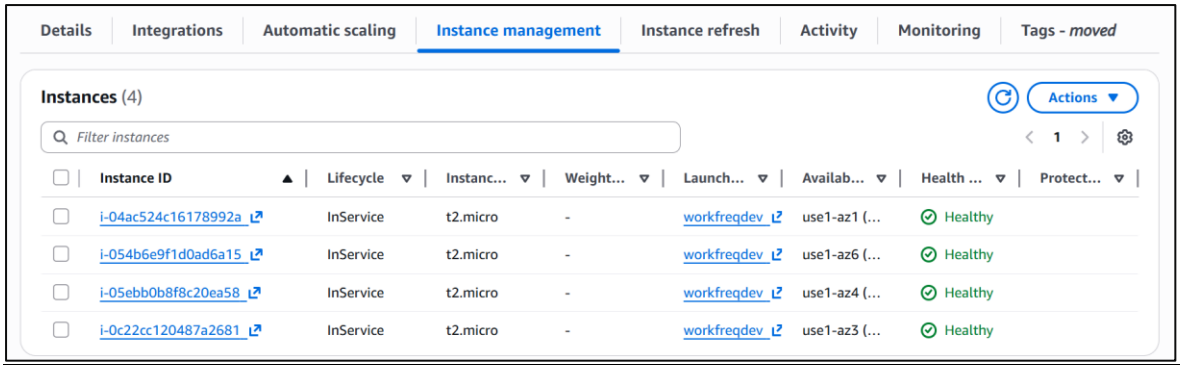

Fig 24: Instances launched by Auto Scaling Group

- **EC2 Instance Dashboard**

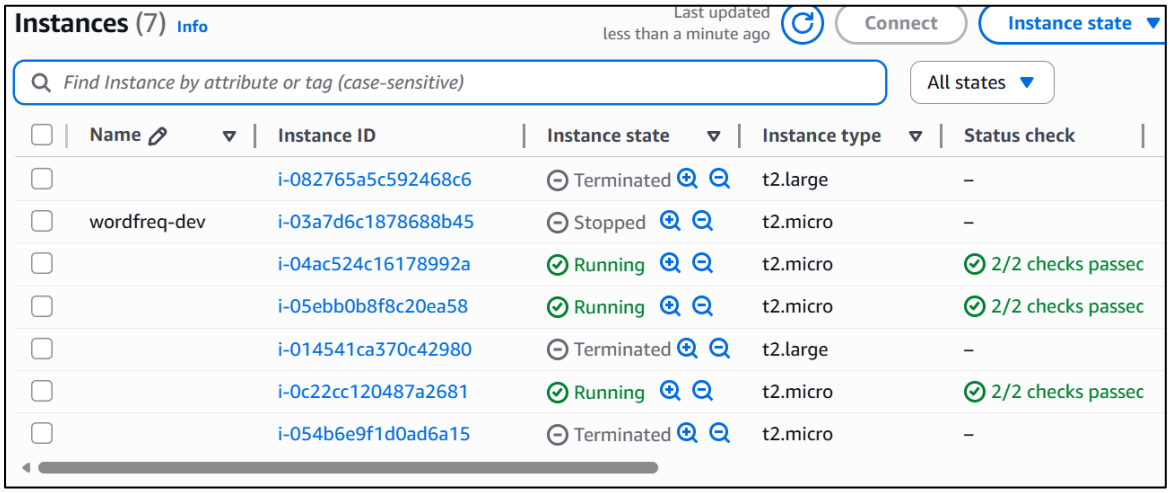

Fig 25: EC2 Instance Dashboard

- **DynamoDB Output**

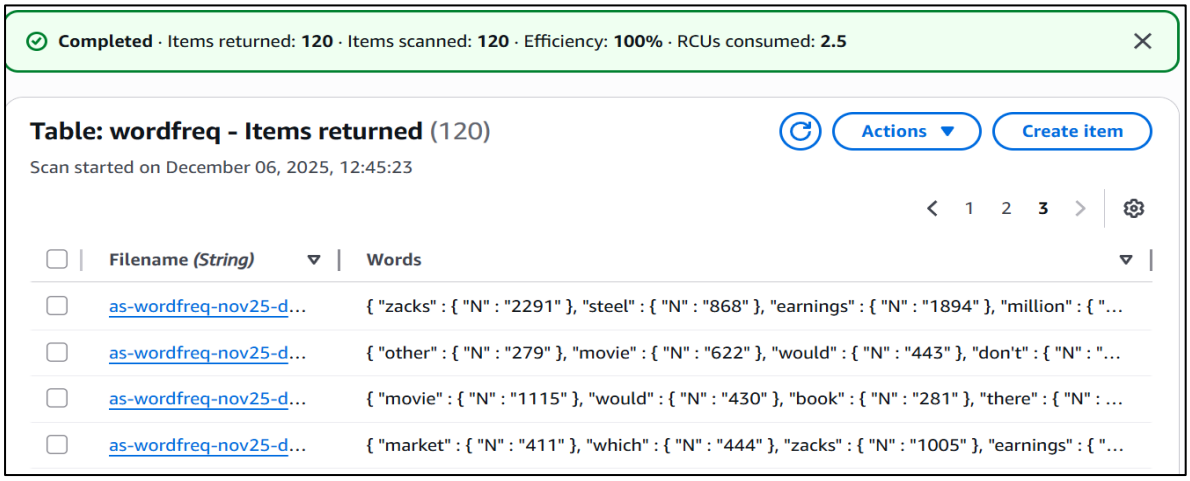

Fig 26: DynamoDB Table

Impact of Scaling Operation on File Processing time:

- **Scale Out Operation:**
  The following settings remained fixed during the tests:
  1. The Cooldown Period: 2 minutes
  2. Instance Type: t2.micro
  3. Cloud Period Time: 1 minute

Table1: File Processing Time vs. Scaling-Out Capacity

S.No	Scaling Action	File Processing Time
1.	1 capacity unit	11min 53 sec
2.	2 capacity units	10 min 15 sec
3.	3 capacity units	8min 30 sec
4.	4 capacity units	8 min 15 sec

Inference:

1. Each instance can process text files independently and adding more instances reduces total processing time by allowing files to be processed in parallel.

2. The fastest file processing time of **8 min 15 sec** was achieved when four capacity units were added.

- **Scale In operation:**
  I set the scaling in policy to remove 2 instances instead of 1 and also reduced the cool down period from 2 minutes to 1 minutes. This led to the faster termination of instances and a cost saving approach.

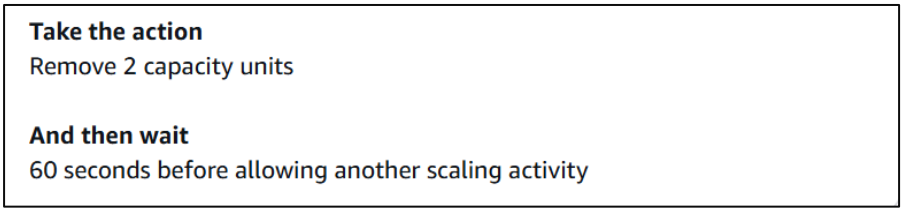

Fig 26: Screenshot of scaling in policy

Impact of Cooldown Period on File Processing Time

The following settings remain fixed during the tests:

1. Scaling Out Action: 3 capacity units
2. Cloud Period Time: 1 minute
3. Instance Type: t2.micro

Table 2: Effect of Cooldown Period on File Processing Time

S.No	Cooldown Period	File Processing Time
1.	2min	8 min 30 sec
2.	1min	7 min 30 sec

Inference:

1. Decreasing the cool down period from 2 min to 1 min allows the ASG to launch new instances more quickly when needed which decreased the file processing time by 1 minute.

Impact of CloudWatch Evaluation Period on File Processing Time

The following settings remain fixed during the tests:

1. Scaling Out Action: 3 capacity units
2. Cooldown Period: 1 min
3. Instance Type: t2.micro

Table 3: Effect of Different Cloud Period Times on File Processing Time

S.No	Cloud Period Time	File Processing Time
1.	1 min	7 min 30 sec
2.	30 sec	7 min 15 sec

Inference:

1. Decreasing the CloudWatch evaluation period from 1 min to 30 Sec reduces the file processing time by 15 sec.
2. Decreasing the cloud watch evaluation time makes ASG group to react faster to the changes in workload.

Impact of Instance Type on File processing Time

The following settings remain fixed during all tests:

1. Scaling Out Action: 3 capacity units
2. Cooldown Period: 1 min
3. Cloud Period Time: 1 min

Table 3: Effect of Instance size on File Processing performance

S.No	Instance Type	File Processing Time
1.	t.small	8 min 30 sec
2.	t.medium	7 min 20 sec
3.	t.large	5 min 50 sec
4.	t.xlarge	3 min 45 sec

Inference:

1. As the instance type gets larger, the file processing time decreases significantly because larger instance type has more CPU, RAM and high computing power which allows faster processing of text file.

Conclusion:

Adding more instances, reducing cooldown period and evaluation periods, and using larger instance types results in system respond faster and reduces file processing time.

Task-D

Objective: Optimise the WordFreq Architecture

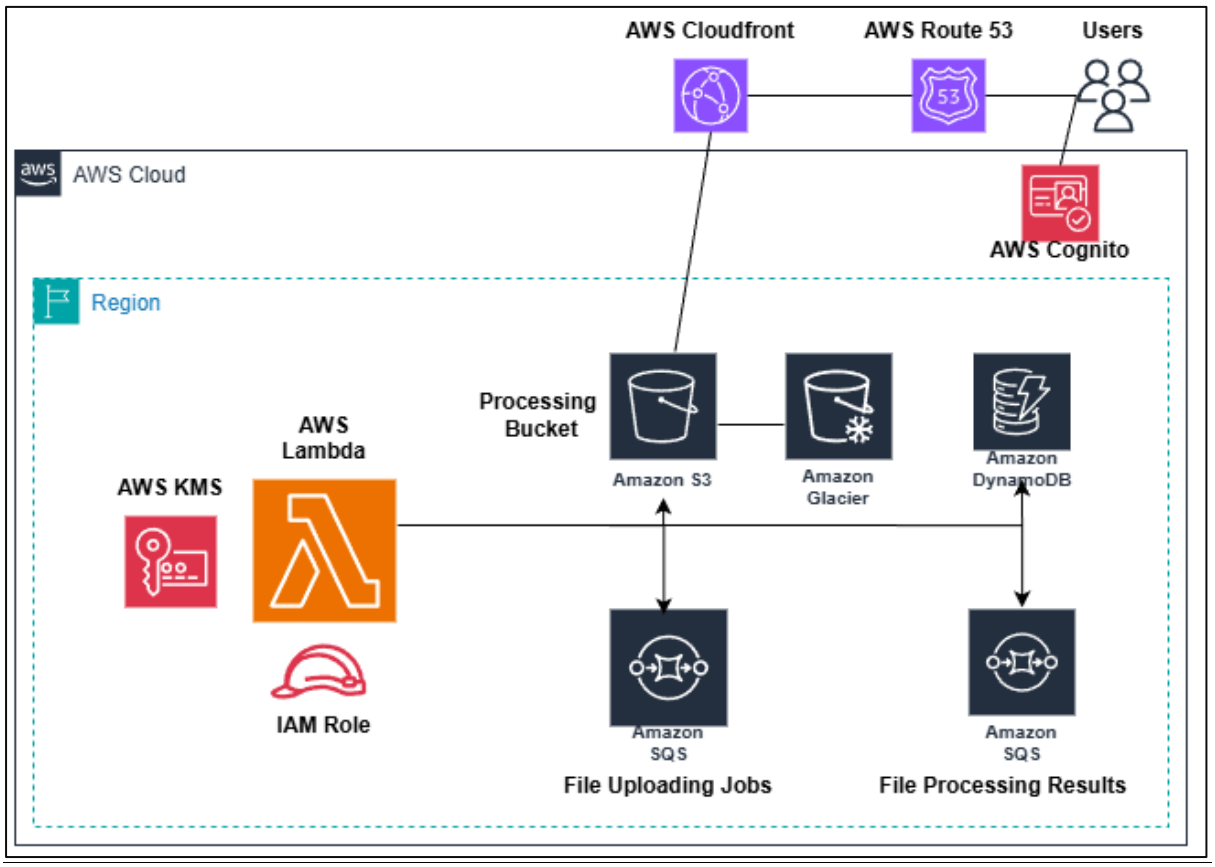

Fig 25: Optimised WordFreq Architecture

Increase Resilience and availability:

- AWS Lambda : It executes the function across multiple Availability Zones. During work load spikes, it scales automatically hence there is no manual intervention and no single point of failure.
- Route 53: Improves availability through redirecting traffic to healthy servers.
- CloudFront: Improves availability through serving cached content from global edge locations.

Long term backups:

- Using S3 versioning and using life cycles rules to move old files to Glacier will back up the data.
- Saving snapshots of DynamoDB table using on demand backup will back up the data.

Cost-effective and efficient application for occasional use:

- Here, AWS Lambda gets triggered by SQS messages and runs only when processing messages. Thus, you pay only for what you use.
- It also scales automatically which is ideal for occasional workloads.

Prevent unauthorised access:

- **AWS Cognito:** It helps manage user authentication and authorization.
- **IAM Role:** It decides what all services Lambda can access.
- **AWS KMS:** It ensures the data is securely encrypted, preventing unauthorized access.

Note: For increasing resilience and global availability further, Lambda function can be deployed in multiple regions along with cross region replication of S3 buckets and DynamoDB

Task-E

Objective: Further improvement

Two alternative data processing services are:

1. Hadoop on Amazon EMR
2. Apache on Amazon EMR

Limitation of the Current WordFreq System:

The architecture of WordFreq system works but it has its limitation for **large scale or multiple file processing**. Due to large dependencies of this architecture on multiple AWS services like SQS, DynamoDB and S3 buckets, a **network overhead and latency are introduced**. EC2 process files one by one, so **throughput is also limited**. Additionally, handling multiple AWS services **increases operational complexity** because all components need to configured for smooth execution.

Advantages of Hadoop on Amazon EMR

The limitations above stated can be addressed using Hadoop on Amazon EMR. In Hadoop, **the data is stored across multiple nodes in HDFS** and nodes read files from their local disks which is **fast and frequent and reduces network overhead** [1]. **Parallel processing** of multiple files can also be allowed using Hadoop's Map Reduce framework which will enable WordFreq to scale horizontally using large datasets [2]. Auto scaling and failure recovery is supported by Amazon EMR which **reduces operational complexity** [2].

Advantages of the Apache on Amazon EMR

It has **in - memory processing** that keeps intermediate data in RAM. It does not have to write back to disk repeatedly which **saves time**. **Improves parallelism** by dividing data into **partitions** and they are processed parallelly in memory across **executors**. Failed tasks are automatically reassigned and are recomputed from other nodes [3]. Querying ranking word frequencies become **easier using Hive** without writing complex codes [4]. Overall, using these tools makes the WordFreq application **quicker, reliable, improves parallelism with less manual intervention.**

Bibliography

[1] "Apache Hadoop on Amazon EMR," *Amazon Web Services, Inc.* https://aws.amazon.com/emr/features/hadoop/

[2] J. Dean and S. Ghemawat, "MapReduce," *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, Jan. 2008, doi: 10.1145/1327452.1327492.

[3] "Overview - Spark 4.0.1 Documentation." https://spark.apache.org/docs/latest/

[4] A. Thusoo *et al.*, "Hive," *Proceedings of the VLDB Endowment*, vol. 2, no. 2, pp. 1626–1629, Aug. 2009, doi: 10.14778/1687553.1687609.

AWS Academy Account Credentials:

Username: fp25098@bristol.ac.uk

Password: Amansingh@7045